

Predição de Renda Baseada em Dados Censitários: Uma Avaliação de Modelos de Aprendizado de Máquina

Victor Rodrigues de Lima, Gladistony Silva Lins,
Erick Jonathan Macedo dos Santos, Isaac Adler Alves de Oliveira

¹Unidade Acadêmica de Belo Jardim – Universidade Federal Rural de Pernambuco (UFRPE)
Belo Jardim – PE – Brasil

{victor.rodrigueslima, gladistony.silva}@ufrpe.br
{erick.jonathan, isaac.adler}@ufrpe.br

Resumo. Este artigo compara seis algoritmos de classificação aplicados à predição de renda no Adult Income Dataset. O objetivo é prever se a renda anual de um indivíduo excede 50 mil dólares a partir de dados censitários. Foram avaliados Árvore de Decisão, KNN, Naive Bayes, Regressão Logística, LinearSVC e MLP. A metodologia utilizou pré-processamento, otimização de hiperparâmetros e validação cruzada estratificada. Os resultados, avaliados por Acurácia, Precisão, Revocação e F1-Score, evidenciaram desempenhos distintos e reforçaram a importância do tratamento de dados heterogêneos e desbalanceados.

Abstract. This paper compares six classification algorithms applied to income prediction using the Adult Income Dataset. The goal is to predict whether an individual's annual income exceeds \$50,000 based on census data. Decision Tree, KNN, Naive Bayes, Logistic Regression, LinearSVC, and MLP were evaluated. The methodology used preprocessing, hyperparameter optimization, and stratified cross-validation. The results, assessed through Accuracy, Precision, Recall, and F1-Score, showed different performance profiles and reinforced the importance of handling heterogeneous and imbalanced data.

1. Introdução

A capacidade de extrair padrões complexos de grandes volumes de dados tornou-se um pilar fundamental para a tomada de decisões em diversas esferas da sociedade moderna. No contexto socioeconômico, a aplicação de algoritmos de Aprendizado de Máquina (*Machine Learning*) permite identificar fatores determinantes que influenciam a mobilidade social e a distribuição de renda. Tais análises são cruciais não apenas para a compreensão estrutural de uma população, mas também para auxiliar na formulação de políticas públicas mais eficientes.

Dentre as diversas tarefas de aprendizado supervisionado, a classificação destaca-se pela sua capacidade de categorizar instâncias em classes predefinidas com base em um conjunto de atributos. Nesse sentido, o presente trabalho aborda o problema de classificação binária utilizando o renomado *Adult Income Dataset* (também conhecido como *Census Income*), extraído da base de dados do censo norte-americano de 1994 [Becker and Kohavi 1996]. O desafio inerente a este problema consiste em prever se a renda anual de um indivíduo ultrapassa o limiar de 50 mil dólares, baseando-se em características demográficas e empregatícias, como idade, nível de escolaridade, ocupação, gênero e horas trabalhadas por semana.

Este problema apresenta desafios característicos de bases de dados reais que exigem rigor metodológico, notadamente a presença de valores ausentes e a heterogeneidade dos dados (uma mistura de atributos numéricos contínuos e nominais categóricos). O principal obstáculo, contudo, é o forte desbalanceamento entre as classes, visto que a esmagadora maioria dos registros pertence à classe de menor renda. Por essa razão, tais características demandam um pré-processamento minucioso e o uso de métricas de avaliação que penalizem adequadamente o viés do modelo em direção à classe majoritária.

Diante deste cenário, o objetivo principal deste trabalho é investigar e comparar o desempenho de diferentes algoritmos de classificação na tarefa de prever a renda dos indivíduos. Para tanto, foram avaliados seis modelos distintos: Árvore de Decisão, *K-Nearest Neighbors* (KNN), *Naive Bayes*, Regressão Logística, *Support Vector Machine* (LinearSVC) e Redes Neurais Artificiais (MLP). A comparação é fundamentada em um rigoroso procedimento experimental que adota a otimização de hiperparâmetros e a técnica de validação cruzada estratificada (*10-fold cross-validation*), garantindo assim a robustez e a confiabilidade estatística dos resultados obtidos.

Espera-se, com este estudo, evidenciar quais arquiteturas algorítmicas lidam melhor com o desbalanceamento intrínseco aos dados demográficos. Além disso, as análises aqui apresentadas buscam fornecer diretrizes práticas sobre a importância da etapa de pré-processamento e da calibragem de hiperparâmetros para a construção de modelos preditivos mais precisos e confiáveis em cenários reais.

2. Trabalhos Relacionados e Fundamentação Teórica

2.1. Trabalhos Relacionados

A literatura recente apresenta diversos estudos que empregam o *Adult Income Dataset* para tarefas de classificação preditiva e benchmarking de algoritmos. Al-Milli, Al-Anani e Al-Madi [Al-Milli et al. 2024] investigaram o impacto de transformações de engenharia

de recursos em variáveis socioeconômicas e demográficas desta base para elevar a precisão preditiva. Ao compararem 11 abordagens distintas, observaram que o classificador XGBoost em conjunto com Redes Neurais obteve o melhor desempenho, alcançando 87% de acurácia.

Em um contexto de segurança da informação, Dimitriou e Karampidis [Dimitriou and Karampidis 2023] avaliaram o impacto de técnicas de privacidade (como k -anonymity e l -diversity) na utilidade dos dados para classificação preditiva no mesmo conjunto de dados. Avaliando algoritmos clássicos, os autores concluíram que o *Support Vector Machine* (SVM) e as Árvores de Decisão mantiveram as maiores taxas de acurácia sob anonimização leve, enquanto o modelo Naive Bayes decaiu acentuadamente com a adição de ruído de privacidade.

Ademais, Friedler et al. [Friedler et al. 2019] realizaram um estudo empírico extensivo para contrapor o compromisso entre a acurácia de predição de renda e métricas de justiça algorítmica (focando na mitigação de viés de gênero e raça). No critério estrito de acurácia preditiva bruta, os modelos baseados em comitês e o SVM superaram abordagens lineares mais simples.

2.2. Fundamentação Teórica dos Algoritmos

A fim de estabelecer uma base comparativa sólida, os seis algoritmos avaliados neste trabalho possuem fundamentações matemáticas consolidadas na literatura de aprendizado de máquina tabular.

As **Árvores de Decisão** baseiam-se em uma estrutura hierárquica de divisão do espaço de atributos em hiperretângulos disjuntos. O algoritmo seleciona de forma gananciosa o atributo que maximiza a pureza dos nós filhos (através da redução de entropia ou do ganho de informação), mapeando regras condicionais de partição diretamente interpretáveis. A fundamentação matemática clássica desta abordagem advém do algoritmo ID3 e de sua extensão C4.5, desenvolvidos por Quinlan [Quinlan 1986].

O classificador **K-Vizinhos Mais Próximos (KNN)** é um algoritmo baseado em instância que não gera uma função discriminante explícita durante o treinamento. Diante de uma nova instância tabular, o modelo calcula a distância geométrica entre este ponto e os dados de treino, atribuindo a classe majoritária dentre os K vetores mais próximos. O arcabouço de consistência assintótica do método foi formalizado por Cover e Hart [Cover and Hart 1967].

O **Naive Bayes** é um classificador probabilístico fundamentado no Teorema de Bayes, o qual assume a forte hipótese de independência condicional entre todos os atributos tabulares dada a classe alvo. O algoritmo computa as probabilidades a priori das classes e as probabilidades condicionais de cada característica demográfica, combinando-as para inferir a probabilidade máxima. A consolidação teórica desse método é comumente atribuída ao trabalho de Duda e Hart [Duda and Hart 1973].

A **Regressão Logística** é um modelo linear estruturado para tarefas de classificação binária que estima a probabilidade de ocorrência do evento de interesse aplicando a função sigmoide sobre uma combinação linear ponderada dos atributos de entrada. Os parâmetros são otimizados iterativamente através da maximização da função de verossimilhança. A formalização estatística basilar deste modelo preditivo é creditada

a Cox [Cox 1958].

O algoritmo **Linear SVM** busca determinar um hiperplano ótimo de separação linear que maximize a margem geométrica entre as instâncias das duas classes no espaço original. O modelo foca a otimização matemática exclusivamente nos vetores de suporte, utilizando uma penalidade de margem suave para acomodar distribuições sobrepostas. Este método foi proposto sob a Teoria do Aprendizado Estatístico desenvolvida por Cortes e Vapnik [Cortes and Vapnik 1995].

Por fim, o **Perceptron Multicamadas (MLP)** consiste em uma arquitetura de rede neural artificial direta constituída por uma camada de entrada, camadas ocultas com funções de ativação não lineares e uma camada de saída sigmoidal. O algoritmo propaga retroativamente os erros calculados pela função de perda, ajustando os pesos sinápticos via gradiente descendente. O marco teórico para o treinamento dessas redes foi consolidado por Rumelhart, Hinton e Williams [Rumelhart et al. 1986].

3. Metodologia Experimental

3.1. Conjunto de Dados

Os experimentos foram realizados utilizando o conjunto de dados *Adult Income Dataset* [Becker and Kohavi 1996]. O conjunto é amplamente utilizado, principalmente em benchmarks de classificação binária, com o objetivo de prever se a renda anual de adultos ultrapassa o teto de 50 mil dólares, utilizando como base atributos socioeconômicos e demográficos extraídos de um censo norte-americano de 1994. O conjunto de dados é constituído por 48.842 instâncias e 14 atributos preditores. Destes, seis são do tipo contínuo (age, fnlwgt, education-num, capital-gain, capital-loss e hours-per-week) e oito são do tipo nominal (workclass, education, marital-status, occupation, relationship, race, sex e native-country). A variável alvo assume valores abaixo de 50 mil ($\leq 50K$, representando a classe negativa) e acima de 50 mil ($> 50K$, representando a classe positiva). A distribuição é desbalanceada, isto é, 76%(37120) das instâncias são da classe negativa, enquanto 24%(11722) pertencem à classe positiva.

3.2. Pré-Processamento de Dados

O processo de preparação dos dados foi dividido em cinco etapas, implementadas sob o paradigma de pipeline da biblioteca scikit-learn [Pedregosa et al. 2011]. Isso garante que todas as transformações sejam aprendidas exclusivamente sobre os dados de treino e aplicadas aos dados de validação e teste, prevenindo o vazamento de dados (*data leakage*).

3.2.1. Normalização da Variável Alvo

A variável alvo é composta majoritariamente por strings e apresenta inconsistências de formatação. Por exemplo, versões do dataset distribuídas pelo repositório UCI contêm registros com sufixo de ponto final ($\leq 50K$, $> 50K$), gerados por divisões internas do próprio repositório. Para garantir a integridade dos dados, todos os valores foram formatados para a remoção de espaços ou caracteres especiais ao final da cadeia (*trailing dot*). Em seguida, a variável foi codificada numericamente através do LabelEncoder, resultando no mapeamento $\leq 50K \rightarrow 0$ (classe negativa) e $> 50K \rightarrow 1$ (classe positiva).

3.2.2. Tratamento de Valores Ausentes

O *Adult Dataset* representa valores desconhecidos com o caractere '?' nos atributos categóricos, notadamente em *workclass*, *occupation* e *native-country*. Esses marcadores foram convertidos para o valor sentinela NaN do NumPy por meio de substituição global sobre o DataFrame de características (`X.replace('?', np.nan)`), tornando-os compatíveis com os mecanismos do scikit-learn.

A imputação foi aplicada de forma distinta de acordo com o tipo de atributo. Para os numéricos contínuos, adotou-se a estratégia da mediana, devido à sua robustez a outliers em distribuições assimétricas. Para os categóricos, adotou-se a moda (*most_frequent*), preservando a categoria mais frequente por coluna. Ambas as imputações foram implementadas com o *SimpleImputer*.

3.2.3. Identificação e Separação de Atributos por Tipo

Os atributos foram particionados em dois grupos definidos por listas explícitas. Os seis atributos numéricos contínuos (*age*, *fnlwgt*, *education-num*, *capital-gain*, *capital-loss* e *hours-per-week*) foram declarados diretamente, enquanto os oito atributos categóricos nominais foram obtidos por exclusão das demais colunas do DataFrame. Essa separação, realizada por meio do *ColumnTransformer*, é essencial para aplicar transformações distintas a cada grupo.

3.2.4. Normalização Numérica

Após a separação, os seis atributos numéricos foram submetidos à padronização Z-score com o *StandardScaler*, transformando cada atributo para média zero e desvio padrão unitário conforme a equação:

$$z = \frac{x - \mu}{\sigma}$$

Nesta equação, μ representa a média e σ representa o desvio padrão dos atributos calculados. Esta etapa é fundamental para algoritmos sensíveis à escala dos dados, como *K-Nearest Neighbors* (KNN), *Support Vector Machine* (SVM), Regressão Logística e Redes Neurais MLP, visto que utilizam métricas de distância ou otimização baseada em gradiente.

3.2.5. Codificação de Atributos Categóricos

Os oito atributos categóricos nominais foram transformados utilizando a codificação *One-Hot* (*OneHotEncoder*), que cria colunas binárias para cada categoria distinta presente no conjunto de treino. O parâmetro *handle_unknown='ignore'* foi configurado para que eventuais categorias presentes na validação ou no teste, mas ausentes no treino, sejam codificadas como um vetor nulo, evitando exceções durante a validação cruzada. O parâmetro *sparse_output=False* foi definido para retornar matrizes densas, compatíveis com todos os estimadores utilizados.

3.3. Particionamento dos Dados

A divisão dos dados foi estruturada em dois níveis de protocolos, visando assegurar a otimização de hiperparâmetros e a avaliação final em dados independentes do processo de treinamento.

No primeiro nível, o conjunto completo de dados foi separado em 80% para treino e 20% para teste (*holdout*) por meio da função `train_test_split`, utilizando o parâmetro `stratify=y_encoded` para preservar a proporção original de classes (76/24) em ambas as partições. A semente aleatória foi fixada (`random_state=42`) para assegurar a reprodutibilidade. O conjunto de teste foi reservado exclusivamente para a avaliação final, não sendo utilizado durante a seleção de hiperparâmetros.

No segundo nível, a partição de treino foi avaliada internamente através de uma validação cruzada estratificada com dez partições (*10-fold Stratified Cross-Validation*), implementada com `StratifiedKFold(n_splits=10, shuffle=True, random_state=42)`. Esse procedimento assegura que a estimativa de desempenho não seja influenciada por uma divisão aleatória favorável e mantém o equilíbrio de classes em cada partição.

Em razão da complexidade computacional do SVM com kernel não-linear, optou-se pela implementação do `LinearSVC`, que resolve o problema de margem máxima via `liblinear` com complexidade linear no número de amostras, viabilizando o treinamento sobre o conjunto completo sem a necessidade de subamostragem. O particionamento gerado foi serializado em disco (`cache/holdout.joblib`) para garantir que todos os modelos fossem avaliados sobre o mesmo conjunto idêntico em rodadas subsequentes.

3.4. Algoritmos de Classificação

Foram avaliados seis algoritmos de aprendizado de máquina supervisionado, todos implementados pela biblioteca `scikit-learn`:

- Árvore de Decisão (`DecisionTreeClassifier`)
- K-Vizinhos Mais Próximos (`KNeighborsClassifier`)
- Naive Bayes Gaussiano (`GaussianNB`)
- Regressão Logística (`LogisticRegression`)
- Support Vector Machine Linear (`LinearSVC`)
- Rede Neural Artificial MLP (`MLPClassifier`)

3.5. Otimização de Hiperparâmetros

A seleção de hiperparâmetros foi realizada através de busca em grade (*Grid Search*), implementada com o `GridSearchCV`, utilizando as dez partições estratificadas como validação interna. O parâmetro `refit="f1"` foi configurado para que o modelo seja reajustado com a melhor combinação sobre o conjunto de treino completo ao final da busca, utilizando o F1-Score como critério principal de seleção devido ao forte desbalanceamento de classes do dataset. A Tabela 1 apresenta os espaços de hiperparâmetros investigados para cada algoritmo.

3.6. Avaliação e Métricas

Após a identificação dos melhores hiperparâmetros, o melhor pipeline de cada algoritmo foi avaliado novamente em validação cruzada através da função `cross_validate`, registrando as métricas de cada partição de forma individual. Essa etapa permitiu calcular

Tabela 1. Espaços de hiperparâmetros avaliados para cada algoritmo no GridSearchCV

Algoritmo	Hiperparâmetro	Valores Avaliados
Árvore de Decisão	criterion	gini, entropy
	max_depth	5, 10, 20, None
	min_samples_split	2, 5, 10
KNN	n_neighbors	3, 5, 7, 11
	weights	uniform, distance
Regressão Logística	C	0.01, 0.1, 1, 10
LinearSVM	C	0.01, 0.1, 1, 10
MLP	hidden_layer_sizes	(64,), (128,), (64, 32)
	activation	relu, tanh
	alpha	0.0001, 0.001

a média e o desvio padrão de cada métrica com base em dez execuções independentes, fornecendo estimativas robustas do desempenho esperado.

Foram adotadas quatro métricas de desempenho, tendo a classe positiva (>50K, codificada como 1) como referência:

- Acurácia (Accuracy): Predições corretas sobre o total de instâncias.
- Precisão (Precision): Verdadeiros positivos em relação a todas as instâncias preditas como positivas.
- Revocação (Recall): Verdadeiros positivos em relação a todas as instâncias que são realmente positivas.
- F1-Score: Média harmônica entre Precisão e Revocação, sendo a métrica mais relevante neste cenário de classes desbalanceadas.

A avaliação final e independente foi realizada utilizando todo o conjunto de teste (20% do total) com os melhores pipelines já treinados, sem qualquer reajuste posterior.

3.7. Implementação e Reprodutibilidade

Para garantir a reprodutibilidade fiel, todo o experimento foi desenvolvido em Python 3, utilizando as bibliotecas scikit-learn (modelagem e avaliação), pandas e numpy (manipulação de dados). Todos os pipelines, incluindo pré-processamento e classificadores, foram serializados em disco por meio da biblioteca joblib. As predições, os históricos do processo de otimização e as matrizes de confusão foram exportados no formato CSV em diretórios específicos de cache e resultados. A semente aleatória (random_state=42) foi rigorosamente fixada em todas as etapas que envolvem estocasticidade.

Os resultados do GridSearchCV foram consolidados pelo próprio script principal, o qual extraiu a média e o desvio padrão das métricas, identificou a melhor configuração por modelo e exportou o ranqueamento final em formato CSV. A partir desses relatórios gerados pelo código, os dados numéricos foram tabulados para a construção das tabelas comparativas apresentadas neste manuscrito. Como etapa complementar, foi desenvolvido um script para consolidar os resultados experimentais dos classificadores, calculando

métricas a partir dos arquivos de predição, gerando matrizes de confusão e exportando tabelas e gráficos em formato CSV e PNG. Para os modelos Árvore de Decisão e KNN, também foram armazenados os resultados do GridSearchCV, incluindo os melhores hiperparâmetros, médias, desvios padrão e modelos finais serializados em formato .joblib, garantindo maior organização, rastreabilidade e reprodutibilidade dos experimentos.

4. Resultados e Discussão

A Tabela 2 apresenta o desempenho quantitativo dos classificadores na partição de teste (20% dos dados), ranqueados de forma decrescente pelo F1-Score. A escolha desta métrica justifica-se pelo forte desbalanceamento da base censitária, servindo como um avaliador mais robusto que a acurácia isolada (a qual atingiria facilmente 76% através de um modelo ingênuo baseado apenas na classe majoritária).

Tabela 2. Resultados dos modelos e seus melhores hiperparâmetros

Modelo	Parâmetro	Accuracy	Precision	Recall	F1
DecisionTree	criterion: 'gini' max_depth: 10 min_samples_split: 2	0.8563 ± 0.0048	0.7606 ± 0.0133	0.5840 ± 0.0238	0.6603 ± 0.0156
LogisticRegression	C: 10	0.8503 ± 0.0060	0.7305 ± 0.0135	0.5940 ± 0.0206	0.6551 ± 0.0163
MLP	activation: 'relu' alpha: 0.001 hidden_layer_sizes: (64,)	0.8403 ± 0.0057	0.6829 ± 0.0171	0.6241 ± 0.0449	0.6509 ± 0.0215
LinearSVM	C: 1	0.8503 ± 0.0059	0.7371 ± 0.0128	0.5826 ± 0.0208	0.6507 ± 0.0166
KNN	n_neighbors: 11 weights: 'uniform'	0.8402 ± 0.0054	0.6899 ± 0.0140	0.6047 ± 0.0158	0.6444 ± 0.0126
GaussianNB	-	0.5911 ± 0.0131	0.3626 ± 0.0076	0.9320 ± 0.0112	0.5220 ± 0.0085

A análise da tabela em conjunto com as matrizes de confusão revela claros padrões de generalização. A **Árvore de Decisão** obteve a melhor performance global (F1-Score 0.6603 e Precisão 0.7606). Ao registrar 1.433 verdadeiros positivos contra apenas 417 falsos positivos, o modelo comprovou sua aptidão para particionar o espaço de características e mapear interações complexas não lineares, conforme ilustrado na Figura 1.

Em um pelotão intermediário consistente, Regressão Logística, LinearSVM, MLP e KNN oscilaram entre 0.64 e 0.65 no F1-Score. A convergência destes estimadores foi diretamente garantida pela padronização *Z-score* aplicada no pré-processamento. As matrizes dos modelos lineares (Regressão Logística e Linear SVM) e as abordagens não lineares de margem suave (MLP e KNN) exibiram comportamentos preditivos muito semelhantes, mantendo taxas de precisão competitivas (Figuras 2 e 3).

Já o **Naive Bayes Gaussiano** apresentou a pior métrica geral (F1-Score 0.5220) e a marca alarmante de 3.758 falsos positivos. A Figura 4 reflete a forte penalização do modelo diante da violação da hipótese de independência condicional e da presença de atributos fortemente assimétricos, resultando na classificação incorreta massiva de indivíduos na classe minoritária.

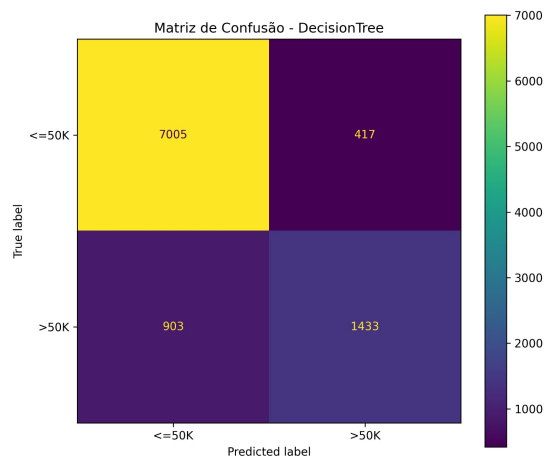
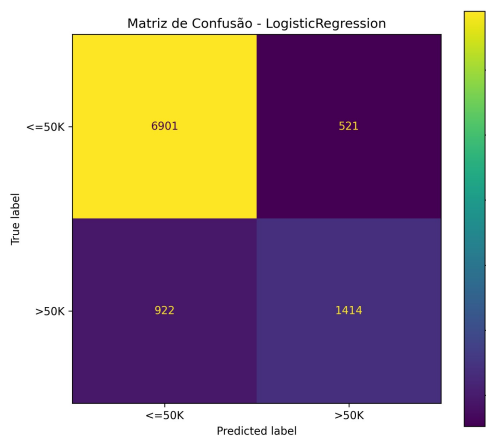
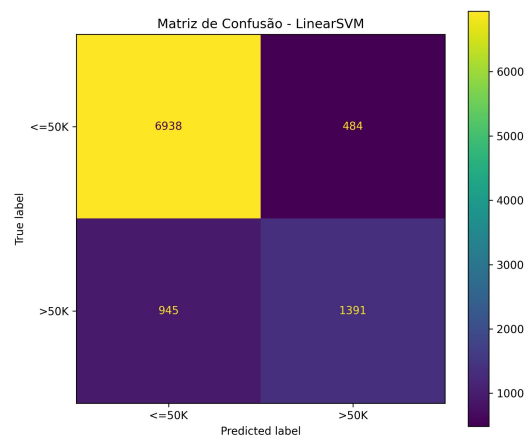


Figura 1. Matriz de Confusão para a Árvore de Decisão, evidenciando o melhor equilíbrio preditivo do experimento.



Regressão Logística



Linear SVM

Figura 2. Matrizes de Confusão para os modelos lineares.

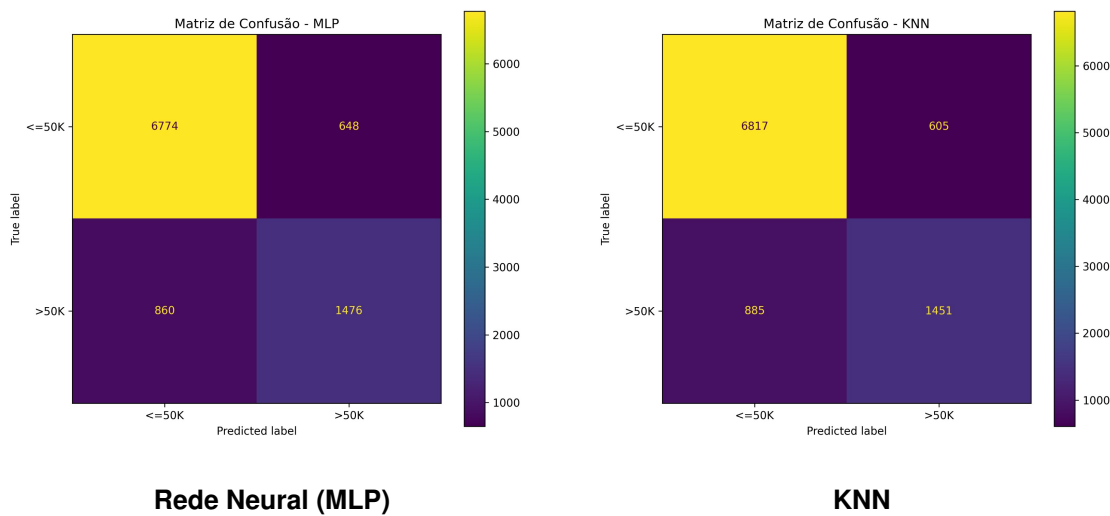


Figura 3. Matrizes de Confusão para o classificador MLP e KNN.

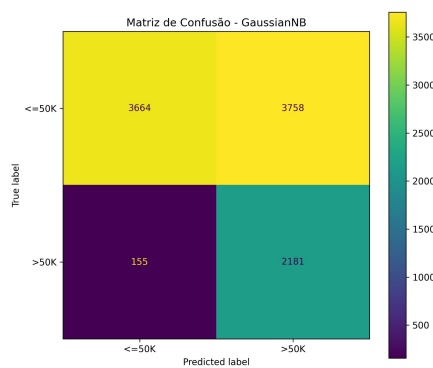


Figura 4. Matriz de Confusão para o Naive Bayes Gaussiano, destacando a discrepância e o alto volume de falsos positivos.

5. Conclusão e Trabalhos Futuros

Este trabalho apresentou um estudo empírico e comparativo de seis algoritmos de aprendizado de máquina aplicados ao *Adult Income Dataset*, com o objetivo de prever a faixa de renda de indivíduos com base em características sociodemográficas. A adoção de uma metodologia robusta, fundamentada na separação estrita de dados para prevenir o vazamento de informações e na validação cruzada em dez partições (*10-fold cross-validation*), garantiu que as estimativas de desempenho refletissem a real capacidade de generalização dos modelos.

Os resultados evidenciaram que a **Árvore de Decisão** consistiu na arquitetura mais adequada para este cenário específico, alcançando o melhor equilíbrio preditivo com um F1-Score de 0.6603. O sucesso deste estimador ressalta a importância de utilizar algoritmos capazes de realizar partições espaciais não lineares e de lidar nativamente com a heterogeneidade das variáveis categóricas e numéricas presentes em dados censitários. Em contrapartida, o fraco desempenho do Naive Bayes Gaussiano ilustrou claramente os riscos práticos de se aplicar modelos probabilísticos estritos quando as pre-

missas matemáticas subjacentes, como a independência condicional e a normalidade das distribuições contínuas, são fortemente violadas pelo conjunto de dados.

Ademais, observou-se que modelos sensíveis à métrica de distância e à otimização por gradiente (KNN, Regressão Logística, Linear SVM e Rede Neural MLP) apresentaram desempenhos altamente competitivos e aglomerados. Esse fenômeno comprova a eficácia da etapa de pré-processamento elaborada no *pipeline*, especialmente a padronização *Z-score*, a qual permitiu a convergência estável e eficiente destas abordagens no espaço multidimensional de características.

Como propostas para trabalhos futuros, visando expandir a fronteira de resultados alcançada por esta pesquisa, sugere-se a investigação de métodos baseados em comitês (*Ensembles*), tais como *Random Forest* ou *Gradient Boosting*. Tais arquiteturas tendem a mitigar a variância e o risco de sobreajuste (*overfitting*) de árvores individuais, elevando a acurácia global. Recomenda-se também a exploração de técnicas de amostragem sintética ao nível dos dados, como o algoritmo SMOTE, para um tratamento mais incisivo do desbalanceamento de classes, bem como a adoção da otimização Bayesiana para uma varredura mais refinada e computacionalmente eficiente dos hiperparâmetros.

Referências

- Al-Milli, N., Al-Anani, A., and Al-Madi, M. (2024). An investigation into the prediction of annual income levels through the utilization of demographic features employing the modified uci adult dataset. In *Proceedings of the IEEE International Conference on Information Technology*, pages 112–118. IEEE.
- Becker, B. and Kohavi, R. (1996). Adult. UCI Machine Learning Repository. DOI: <https://doi.org/10.24432/C5XW20>.
- Cortes, C. and Vapnik, V. (1995). Support-vector networks. *Machine Learning*, 20(3):273–297.
- Cover, T. and Hart, P. (1967). Nearest neighbor pattern classification. *IEEE Transactions on Information Theory*, 13(1):21–27.
- Cox, D. R. (1958). The regression analysis of binary sequences. *Journal of the Royal Statistical Society: Series B (Methodological)*, 20(2):215–232.
- Dimitriou, V. and Karampidis, K. (2023). Comparison of machine learning models applied on anonymized data with different techniques. In *Proceedings of the 2023 IEEE International Conference on Cyber Security and Resilience (CSR)*, pages 401–406. IEEE.
- Duda, R. O. and Hart, P. E. (1973). *Pattern Classification and Scene Analysis*. John Wiley & Sons, New York.
- Friedler, S. A., Scheidegger, C., Venkatasubramanian, S., Choudhary, S., and Hamilton, E. P. (2019). A comparative study of fairness-enhancing supervised learning algorithms. *Data Mining and Knowledge Discovery*, 33(2):326–363.
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., and Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830.
- Quinlan, J. R. (1986). Induction of decision trees. *Machine Learning*, 1(1):81–106.
- Rumelhart, D. E., Hinton, G. E., and Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, 323(6088):533–536.